

Aim

To write a Java program that demonstrates the Deque interface methods `addFirst()`, `addLast()`, `offerFirst()`, `offerLast()`, `removeFirst()`, `removeLast()`, `pollFirst()`, `pollLast()`, `peekFirst()` and `peekLast()`. The program also shows how the exception-throwing methods differ from the special-value-returning methods when the deque is full or empty.

Algorithm

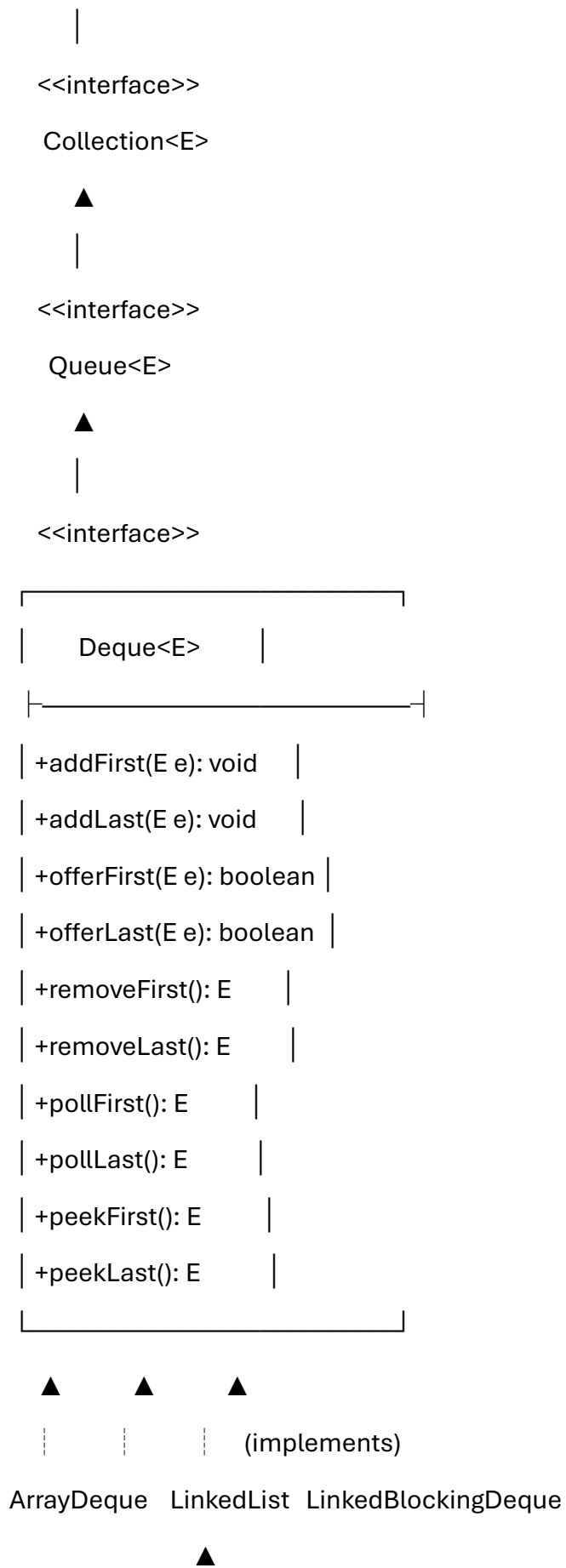
1. Start.
2. Import `java.util.*` and `java.util.concurrent.LinkedBlockingDeque`.
3. Create a `Deque<Integer>` object with a capacity of 3 (`LinkedBlockingDeque`), so the "full" case can be shown.
4. Insert elements using `addFirst(20)`, `addLast(30)` and `addFirst(10)`, then display the deque.
5. Try to insert into the full deque:
 - `addFirst(5)` throws `IllegalStateException`. Catch it and display it.
 - `offerFirst(5)` and `offerLast(40)` return false. Display the results.
6. Use `peekFirst()` and `peekLast()` to view the end elements without removing them.
7. Use `removeFirst()` and `removeLast()` to remove the first and last elements, then display the deque.
8. Use `offerFirst(5)` and `offerLast(40)` to insert again. They now return true.
9. Use `pollFirst()` and `pollLast()` to remove the end elements, then remove the last remaining element with `pollFirst()` so the deque is empty.
10. On the empty deque:
 - `pollFirst()`, `pollLast()`, `peekFirst()` and `peekLast()` return null.
 - `removeFirst()` and `removeLast()` throw `NoSuchElementException`. Catch and display them.
11. Stop.

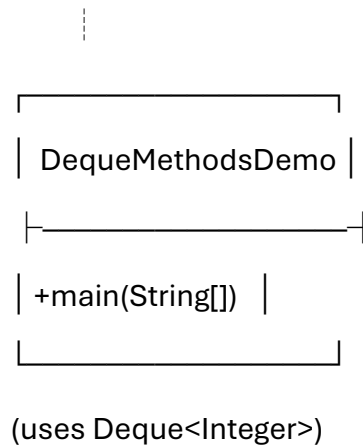
UML Diagram

<<interface>>

Iterable<E>







Code:

```
package Observations;

import java.util.*;
import java.util.concurrent.LinkedBlockingDeque;

public class DequeInterface {
    public static void main(String[] args) {
        // Capacity-restricted deque (capacity = 3)
        Deque<Integer> dq = new LinkedBlockingDeque<>(3);

        // addFirst() and addLast()
        dq.addFirst(20);
        dq.addLast(30);
        dq.addFirst(10);
        System.out.println("After addFirst(20), addLast(30), addFirst(10): " + dq);

        // Deque is full: add throws exception, offer returns false
        try {
            dq.addFirst(5);
        } catch (IllegalStateException e) {
            System.out.println("addFirst(5) -> Exception: " + e);
        }
    }
}
```

```
}

System.out.println("offerFirst(5) : " + dq.offerFirst(5));
System.out.println("offerLast(40) : " + dq.offerLast(40));


// peekFirst() and peekLast()
System.out.println("peekFirst() : " + dq.peekFirst());
System.out.println("peekLast() : " + dq.peekLast());
System.out.println("Deque after peek: " + dq);


// removeFirst() and removeLast()
System.out.println("removeFirst() : " + dq.removeFirst());
System.out.println("removeLast() : " + dq.removeLast());
System.out.println("Deque after remove: " + dq);


// offerFirst() and offerLast() now succeed
System.out.println("offerFirst(5) : " + dq.offerFirst(5));
System.out.println("offerLast(40) : " + dq.offerLast(40));
System.out.println("Deque after offer: " + dq);


// pollFirst() and pollLast()
System.out.println("pollFirst() : " + dq.pollFirst());
System.out.println("pollLast() : " + dq.pollLast());
System.out.println("Deque after poll: " + dq);


// Make the deque empty
dq.pollFirst();
System.out.println("Deque after emptying: " + dq);
```

```
// Empty deque: poll/peek return null, remove throws exception

System.out.println("pollFirst() : " + dq.pollFirst());
System.out.println("pollLast() : " + dq.pollLast());
System.out.println("peekFirst() : " + dq.peekFirst());
System.out.println("peekLast() : " + dq.peekLast());

try {
    dq.removeFirst();
} catch (NoSuchElementException e) {
    System.out.println("removeFirst() -> Exception: " + e);
}

try {
    dq.removeLast();
} catch (NoSuchElementException e) {
    System.out.println("removeLast() -> Exception: " + e);
}

}
```

Output:

```

Observations.DequeInterface
After addFirst(20), addLast(30), addFirst(10): [10, 20, 30]
addFirst(5) -> Exception: java.lang.IllegalStateException: Deque full
offerFirst(5) : false
offerLast(40) : false
peekFirst() : 10
peekLast() : 30
Deque after peek: [10, 20, 30]
removeFirst() : 10
removeLast() : 30
Deque after remove: [20]
offerFirst(5) : true
offerLast(40) : true
Deque after offer: [5, 20, 40]
pollFirst() : 5
pollLast() : 40
Deque after poll: [20]
Deque after emptying: []
pollFirst() : null
pollLast() : null
peekFirst() : null
peekLast() : null
removeFirst() -> Exception: java.util.NoSuchElementException
removeLast() -> Exception: java.util.NoSuchElementException
PS C:\Users\Sahithi\pyla\Desktop\Sahithi_java>

```

Result:

I used `LinkedBlockingDeque` with a fixed capacity because `ArrayDeque` is unbounded and never throws on insertion, which would hide the full-deque behavior. If your syllabus expects `ArrayDeque`, the program still works, but the "full" part of the output (the exception and the false results) won't appear.